

12.2.3 Pertanyaan

1. Jelaskan perbedaan struktur dan mekanisme traversal antara Single Linked List dan Double Linked List!
 - single linked list hanya memiliki 1 tali pengait, sedangkan double linked list punya 2 seperti namanya. single linked list ada next aja, kalau double linked list ada next dan prev. jadi arah traversal untuk single cuma 1 arah saja, sedangkan untuk double memiliki 2 arah, bisa dari depan atau belakang.
2. Perhatikan class Node, di dalamnya terdapat atribut next dan prev. Jelaskan fungsi masing-masing atribut tersebut pada proses traversal dan manipulasi node!
 - next: tali pengait tiap node lurus ke depan
 - prev: tali pengait tiap node lurus ke belakang
 - fungsinya: pemutus dan penyambung relasi/alamat memori serta penjaga hubungan rantai relasi.
3. Perhatikan konstruktor pada class DoubleLinkedList. Jelaskan fungsi konstruktor tersebut terhadap kondisi awal linked list!
 - menentukan nilai head dan tail. Null berarti data belum memiliki nilai dan tidak menunjuk ke alamat node manapun
4. Perhatikan potongan kode berikut:
`if (isEmpty()) { head = tail = newNode; }`
Mengapa head dan tail harus menunjuk node yang sama ketika linked list masih kosong?
 - karena data masih berisi 1 node, jadi head dan tail berada di node yang sama. Pointer next dan prev juga masih bernilai null, karena tidak merujuk ke alamat node manapun.
5. Modifikasi method print() agar menampilkan pesan "Linked List masih kosong" ketika tidak terdapat data pada linked list!

```
public void print(){  
    if (isEmpty()) {  
        System.out.println("Linked List masih Kosong");  
        return;  
    }  
}
```

12.3.3 Pertanyaan Percobaan

1. Perhatikan potongan kode berikut pada method removeFirst():
`head = head.next;`
`head.prev = null;`
Jelaskan fungsi masing-masing statement tersebut pada proses penghapusan node!
 - `head = head.next;` = Memindahkan pointer head ke node setelah head, agar memutus tali silaturahmi dengan node yang akan dihapus.
 - `head.prev = null;` = Mengubah nilai pointer head.prev menjadi null, agar tidak merujuk ke alamat node yang akan dihapus. dengan begitu node yang dihapus sudah tidak dianggap ada, meskipun sebenarnya masih ada.

2. Modifikasi method `removeFirst()` dan `removeLast()` agar program menampilkan data yang berhasil dihapus!

- `removeFisrt()`;

```
public void removeFirst() {
    if (isEmpty()){
        System.out.println(x: "List Kosong");
        return;
    }
    if (head == tail) {
        System.out.println("Data yang berhasil dihapus adalah Mahasiswa dengan Nama: " + head.data.nama);
        head = tail = null;
    } else {
        System.out.println("Data yang berhasil dihapus adalah Mahasiswa dengan Nama: " + head.data.nama);
        head = head.next;
        head.prev = null;
    }
}
```

- `removeLast()`;

```
public void removeLast(){
    if (isEmpty()) {
        System.out.println(x: "List Kosong");
        return;
    }
    if (head == tail) {
        System.out.println("Data yang berhasil dihapus adalah Mahasiswa dengan Nama: " + head.data.nama);
        head = tail = null;
    } else {
        System.out.println("Data yang berhasil dihapus adalah Mahasiswa dengan Nama: " + tail.data.nama);
        tail = tail.prev;
        tail.next = null;
    }
}
```